Lab 04 — Pods, Deployments & Self-Healing Rollouts

ID: K8S-LAB-04 · **Category:** Core · **Level:** Beginner · **Duration:** 60 min

Focus: Pods, ReplicaSets, Deployments, scaling, rolling updates, rollback

Objective

Author a Deployment, scale it, roll out a new image, roll back when it breaks, and watch self-healing in action.

Concepts Covered

• Why you almost never create raw Pods in production

• Rolling update strategy parameters

• How rollback uses the ReplicaSet history

Prerequisites

• A cluster from Lab 1

Architecture

Deployments don't manage Pods directly — they manage ReplicaSets and orchestrate the gradual switch between two of them.

```
Deployment (desired: 3, image: v1)
   |
   v
 ReplicaSet  --owns-->  Pod  Pod  Pod
   |
 (on update)
   v
 New ReplicaSet (v2)  --owns-->  Pod  Pod  Pod
   |
 Old ReplicaSet scaled to 0 (kept for rollback)
```

Lab Steps

01. Create the Deployment

Three replicas of nginx 1.25.

```
cat <<EOF | kubectl apply -f -
apiVersion: apps/v1
kind: Deployment
metadata: {name: web}
spec:
  replicas: 3
  selector: {matchLabels: {app: web}}
  template:
    metadata: {labels: {app: web}}
    spec:
      containers:
      - name: web
        image: nginx:1.25
        ports: [{containerPort: 80}]
```

```
EOF
```

Expected: deployment.apps/web created; 3 pods Running within ~10s.

Internals: Deployment controller creates a ReplicaSet, which creates 3 Pods.

02. Scale up

Edits the ReplicaSet's `replicas` field. Two new Pods get scheduled.

```
kubectl scale deploy/web --replicas=5
kubectl get pods -l app=web
```

Expected: 5/5 ready within ~10s.

Internals: ReplicaSet controller posts new Pods until count == desired.

03. Rolling update

Replaces Pods gradually using maxSurge=25%, maxUnavailable=25% defaults.

```
kubectl set image deploy/web web=nginx:1.27
kubectl rollout status deploy/web
```

Expected: `deployment "web" successfully rolled out`

Internals: A NEW ReplicaSet is created at desired=1, then scaled up while the old is scaled down — never both at full capacity unless surge allows it.

04. Break it on purpose and roll back

Bad image → rollout stalls. `rollout undo` switches traffic back to the previous ReplicaSet.

```
kubectl set image deploy/web web=nginx:doesnotexist
kubectl rollout status deploy/web --timeout=30s || true
kubectl rollout undo deploy/web
kubectl rollout status deploy/web
```

Expected: After undo: rollout succeeds.

Internals: Deployment keeps `revisionHistoryLimit` ReplicaSets at replicas=0 specifically for this.

05. Watch self-healing

Deleting a managed Pod produces an immediate replacement.

```
POD=$(kubectl get pod -l app=web -o name | head -1)
kubectl delete $POD
kubectl get pods -l app=web -w
```

Expected: A new Pod with a different suffix appears within ~2s.

Internals: ReplicaSet controller sees count<desired and POSTs a new Pod.

Common Errors

selector mismatch on a manual edit — Cause: Edited template labels but not the selector (or vice-versa) · Fix: Selectors are immutable on Deployments — delete and recreate

Rollout stuck with maxUnavailable=0 — Cause: No spare capacity to start new Pods · Fix: Raise maxSurge or add a node

Troubleshooting

• `kubectl rollout history deploy/web` shows every revision

• `kubectl rollout undo deploy/web --to-revision=2` jumps to a specific revision

Enterprise Relevance

Rolling updates + rollback are the cheapest production safety net you can buy. Every deploy should be undoable in one command.

Validation

• You can update an image without downtime

• You can roll back a bad deploy in under 30 seconds

Related Incidents & Guide Sections

Incidents: K8S-INC-3001

• Pod lifecycle & probes

Questions

Beginner

Q. Why prefer Deployments over raw Pods?

A. Self-healing, scaling, and rollouts.

Intermediate

Q. What does maxUnavailable control?

A. How many Pods can be down at once during a rollout.

Advanced

Q. Why are Deployment selectors immutable?

A. Changing them would orphan the existing ReplicaSet's Pods, risking double-running or no-running states.

Scenario

Q. A rollout is wedged. Old pods are gone, new pods Pending. What broke?

A. Likely a scheduling constraint introduced in the new spec (resources, affinity, node selector).

Viva

Q. What chain owns a Pod created by a Deployment?

A. Deployment → ReplicaSet → Pod, tracked via ownerReferences.